



WHY REAL-TIME MAKES BATCH DATA MODELING HARDER THAN EXPECTED: LESSONS FROM A PERSONAL DATA ENGINEERING PROJECT

Yunata Gunawan
Data Engineer
August 5, 2026



BACKGROUND

ACROSS EVERY INDUSTRY, BUSINESSES ARE **TRANSFORMING THE WAY THEY USE DATA**. BECOMING FASTER, MORE AGILE, AND BETTER EQUIPPED TO DELIVER MEANINGFUL RESULTS.

THE CHALLENGE: TRANSITIONING FROM BATCH-BASED DECISION SYSTEMS TO REAL-TIME ANALYTICS

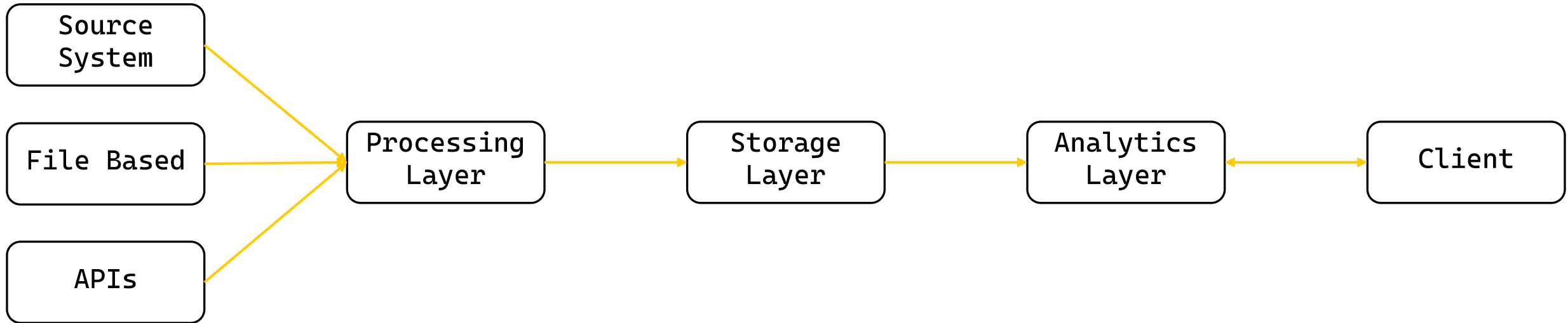
- Legacy enterprise decision systems rely on scheduled batch ETL pipelines
- Data is processed periodically, creating latency between events and insights
- BI and decision-making depend on delayed historical data

**IF THERE ARE USE CASES THAT
REQUIRE COMBINING BATCH
AND STREAM PROCESSING,
CAN OUR EXISTING BATCH
SYSTEM SUPPORT REAL-TIME
BEHAVIOR?**

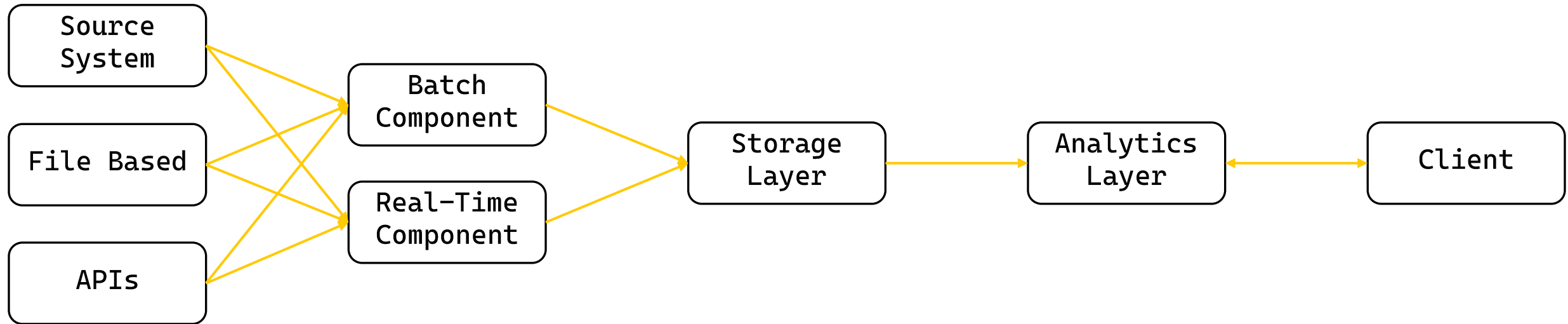
ANSWER: EVOLVE TOWARD A HYBRID BATCH + STREAMING ARCHITECTURE

- Keep batch processing for historical analytics and large-scale computation
- Add streaming capabilities for real-time data processing
- Combine both paths through a unified data platform

TRADITIONAL BATCH PIPELINE



ADDING REAL-TIME CAPABILITIES



REAL-TIME IS MORE THAN ADDING A NEW COMPONENT

Data Model

Batch:

- Snapshot-based
- Current state
- Aggregated views

Real-time:

- Change-based
- Event history
- Rebuildable state

Processing

Batch:

- Scheduled jobs
- Process all data
- High latency

Real-time:

- Event-driven
- Continuous flow
- Handle ordering
- Handle duplicates

IMPACT?

(Source View)

Orders Table

ID	Order Name	Order Date	Status

The first major impact of moving toward real-time is the data ingestion and data modeling layer

(Batch View)

Snapshot, Delete/Truncate
& Insert

ID	Order Name	Order Date	Status

(Real-Time View)

Event Model, Append Only

ID	Order Name	Order Date	Status	Event Date

State Model, Merge

ID	Order Name	Order Date	Status	Event Date

CURRENT ASSUMPTION

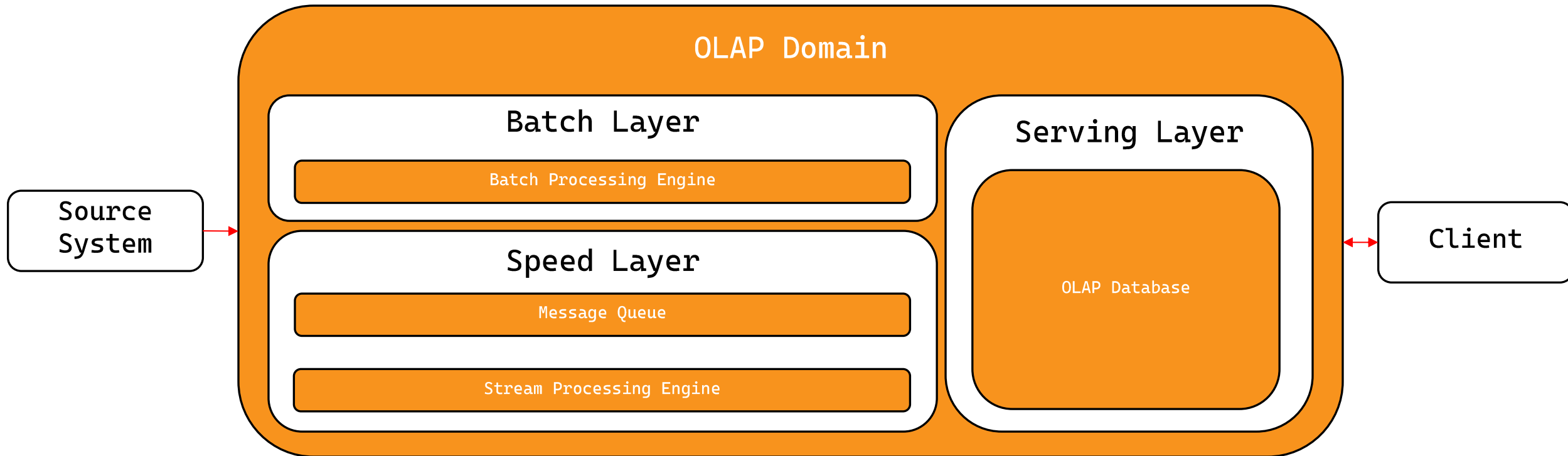
Batch Data Ingestion & Processing

- Simple
- Single source of truth
- Full snapshot
- Predictable
- Error? Simply rerun (transient)

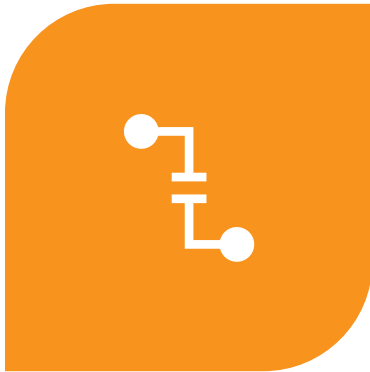
Data Ingestion & Processing with Batch and Real-Time

- Operational
- Maintainability
- Idempotency
- Ordering
- Duplicate handling
- Late events
- Replay capability
- State management

DESIGN OVERVIEW: LAMBDA ARCHITECTURE



DESIGN CONSIDERATION: PROCESS



BATCH AND STREAM PIPELINES REMAIN ISOLATED TO MAINTAIN SEPARATION OF CONCERNS UNTIL FINAL AGGREGATION IN THE SERVING LAYER

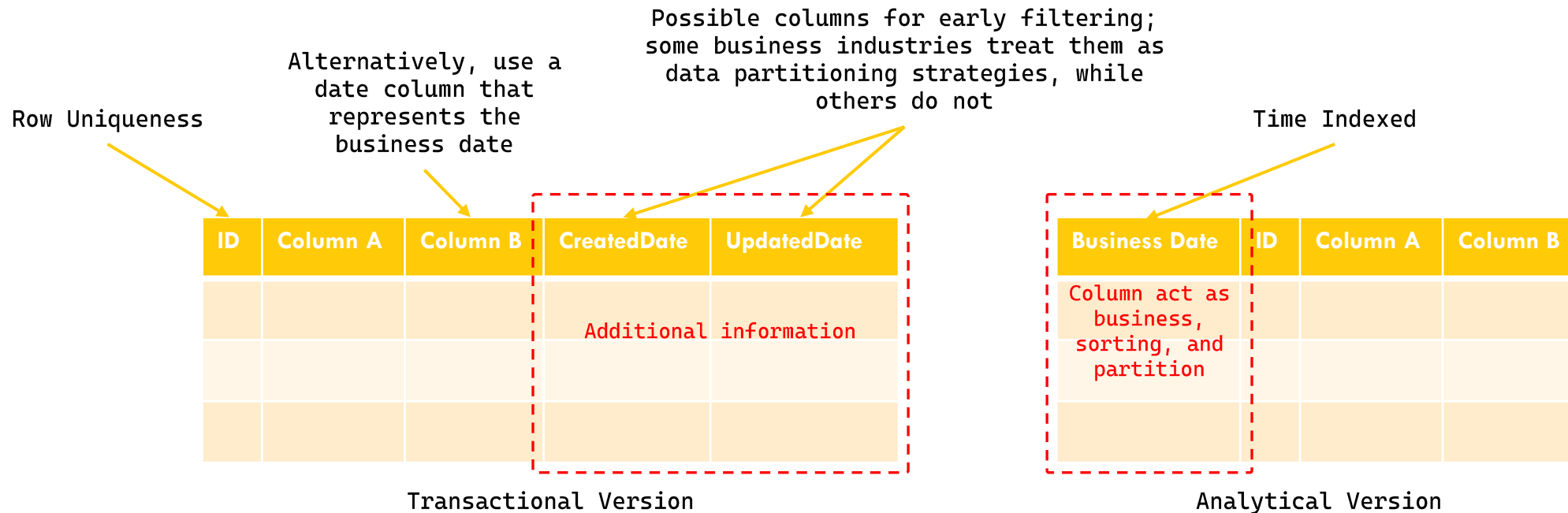


BATCH PROCESSING SUPPORTS RECONCILIATION AND HISTORICAL ANALYSIS, WHILE THE SPEED PATH ENABLES REAL-TIME ANALYSIS



BATCH AND REAL-TIME DATA MERGING CAN BE HANDLED THROUGH AUTOMATIC OR MANUAL PROCESSES

DESIGN CONSIDERATION: THE DATA ITSELF



Note:

- The example Analytics Data Model is daily time-indexed, with each row representing the business state for a specific day

DATA FLOW

Orders Table

ID	Name	Order Date	Status

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources

Real-time/Stream

Orders Staging/Bronze

Business Date	ID	Name	Order Date	Status

Orders Historical/Silver

Business Date	ID	Name	Order Date	Status

Batch

Batch

INITIAL STATE: EMPTY

Batch Date: 2026-07-01
Current Date: 2026-07-02
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources

INITIAL DATA LOAD TO SOURCE

Batch Date: 2026-07-01
Current Date: 2026-07-02
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-01	PENDING

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources

INITIAL STREAM DATA CAPTURE AND PROCESSING

Batch Date: 2026-07-01
Current Date: 2026-07-02
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-01	PENDING

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM

ADDING NEW SAMPLE DATA

Batch Date: 2026-07-01
Current Date: 2026-07-02
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-01	PENDING
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM

STREAM DATA CAPTURE

Batch Date: 2026-07-01
Current Date: 2026-07-02
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-01	PENDING
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM
2026-07-02	2	B	2026-07-01	DELIVERED	2026-07-02	STREAM

END-OF-DAY BATCH JOB EXECUTION

Batch Date: 2026-07-01
Current Date: 2026-07-02
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-01	PENDING
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM
2026-07-02	2	B	2026-07-01	DELIVERED	2026-07-02	STREAM
2026-07-01	2	B	2026-07-01	DELIVERED	NULL	BATCH
2026-07-01	1	A	2026-07-01	PENDING	NULL	BATCH

NEXT-DAY DATA UPDATE

Batch Date: 2026-07-02
Current Date: 2026-07-03
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-02	PROGRESS
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM
2026-07-02	2	B	2026-07-01	DELIVERED	2026-07-02	STREAM
2026-07-01	2	B	2026-07-01	DELIVERED	NULL	BATCH
2026-07-01	1	A	2026-07-01	PENDING	NULL	BATCH

STREAM RECAPTURES THE DATA

Batch Date: 2026-07-02
Current Date: 2026-07-03
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-02	PROGRESS
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM
2026-07-02	2	B	2026-07-01	DELIVERED	2026-07-02	STREAM
2026-07-01	2	B	2026-07-01	DELIVERED	NULL	BATCH
2026-07-01	1	A	2026-07-01	PENDING	NULL	BATCH
2026-07-03	1	A	2026-07-02	PROGRESS	2026-07-03	STREAM

END-OF-DAY BATCH JOB EXECUTION WITH NO FURTHER UPDATES

Batch Date: 2026-07-02
Current Date: 2026-07-03
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-02	PROGRESS
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM
2026-07-02	2	B	2026-07-01	DELIVERED	2026-07-02	STREAM
2026-07-01	2	B	2026-07-01	DELIVERED	NULL	BATCH
2026-07-01	1	A	2026-07-01	PENDING	NULL	BATCH
2026-07-03	1	A	2026-07-02	PROGRESS	2026-07-03	STREAM
2026-07-02	1	A	2026-07-02	PROGRESS	NULL	BATCH
2026-07-02	2	B	2026-07-01	DELIVERED	NULL	BATCH

SIMILAR DATA BASED ON SORTING KEY WAS MERGED

Batch Date: 2026-07-02
Current Date: 2026-07-03
Ordering / Sorting Key: Business Date, ID

Orders Table

ID	Name	Order Date	Status
1	A	2026-07-02	PROGRESS
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-02	1	A	2026-07-01	PENDING	2026-07-02	STREAM
2026-07-02	2	B	2026-07-01	DELIVERED	2026-07-02	STREAM
2026-07-01	2	B	2026-07-01	DELIVERED	NULL	BATCH
2026-07-01	1	A	2026-07-01	PENDING	NULL	BATCH
2026-07-03	1	A	2026-07-02	PROGRESS	2026-07-03	STREAM
2026-07-02	1	A	2026-07-02	PROGRESS	NULL	BATCH
2026-07-02	2	B	2026-07-01	DELIVERED	NULL	BATCH

FINAL RESULTS

Batch Date: 2026-07-02
Current Date: 2026-07-03
Ordering / Sorting Key: Business Date, ID

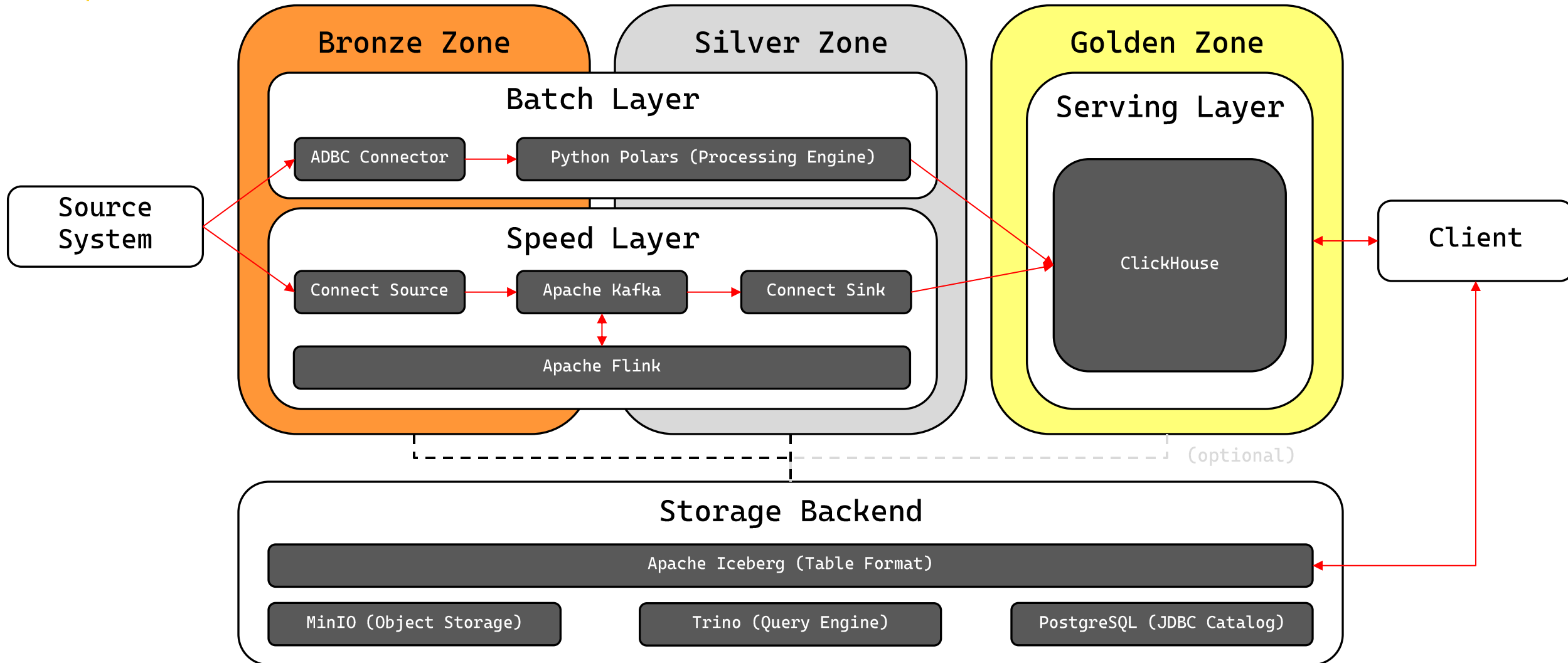
Orders Table

ID	Name	Order Date	Status
1	A	2026-07-02	PROGRESS
2	B	2026-07-01	DELIVERED

Orders Curated/Golden

Business Date	ID	Name	Order Date	Status	Event Time	Sources
2026-07-01	2	B	2026-07-01	DELIVERED	NULL	BATCH
2026-07-01	1	A	2026-07-01	PENDING	NULL	BATCH
2026-07-03	1	A	2026-07-02	PROGRESS	2026-07-03	STREAM
2026-07-02	1	A	2026-07-02	PROGRESS	NULL	BATCH
2026-07-02	2	B	2026-07-01	DELIVERED	NULL	BATCH

PROTOTYPE IMPLEMENTATION ARCHITECTURE

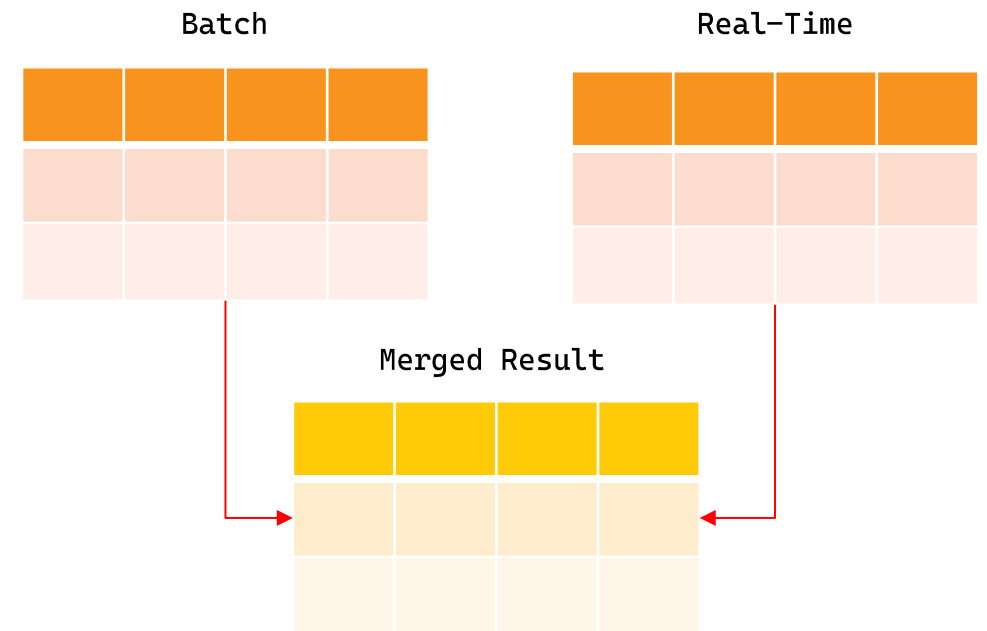


CLICKHOUSE SIMPLIFIES SERVING LAYER IMPLEMENTATION

1. Custom logic for deduplication and merging must be implemented when ClickHouse is not used
2. The custom logic can be executed through scheduled cron jobs
3. Each target table may have different keys that act as unique identifiers
4. Additional TTL (Time-to-Live) processes can be implemented to improve query performance or support retention policies when ClickHouse is not used

Note:

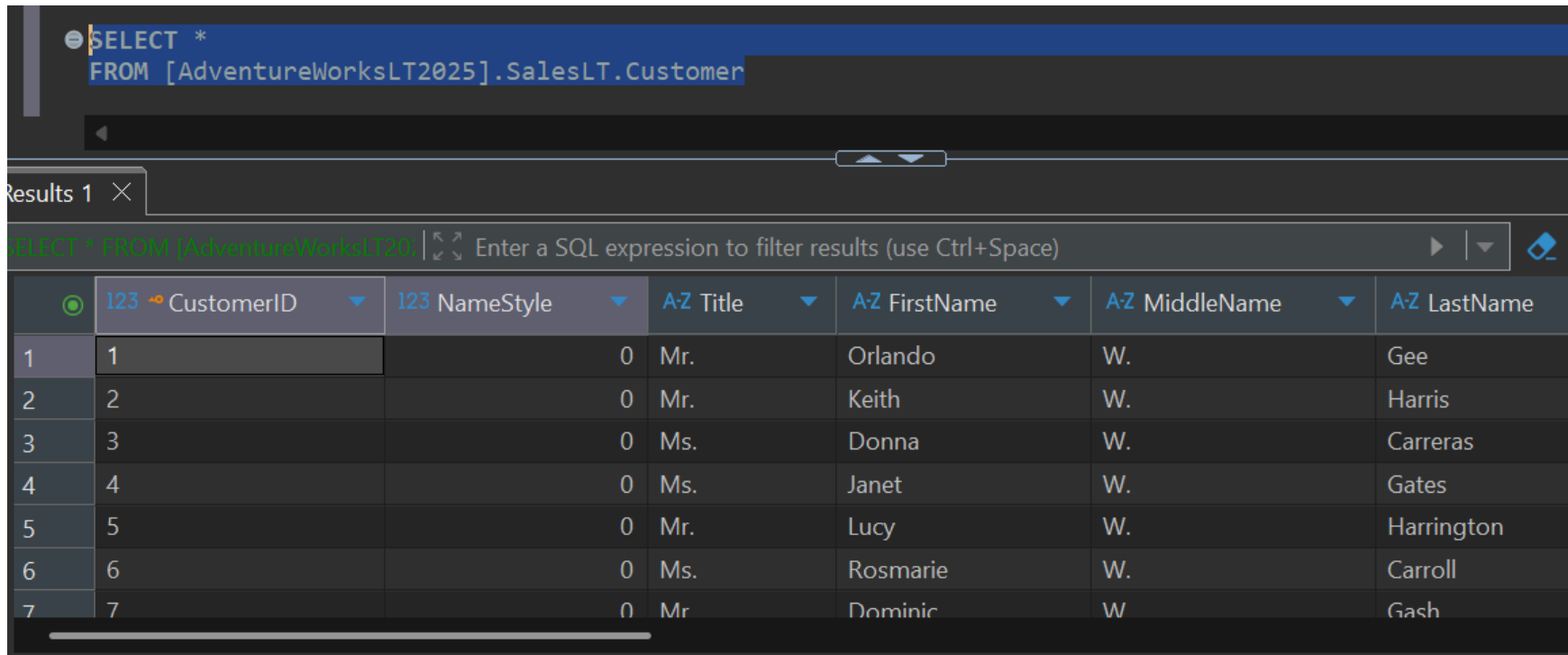
- Reducing the amount of data scanned generally improves query performance



SOURCE DATABASE OVERVIEW

Note:

- Enable CDC for all tables being used



The screenshot shows a SQL Server Enterprise Manager interface. At the top, a query window displays the following SQL statement:

```
SELECT *  
FROM [AdventureWorksLT2025].SalesLT.Customer
```

Below the query window, the 'Results 1' tab is active, showing a table with 7 rows and 7 columns. The columns are: CustomerID, NameStyle, Title, FirstName, MiddleName, and LastName. The first row of data is highlighted in blue.

	123 CustomerID	123 NameStyle	A-Z Title	A-Z FirstName	A-Z MiddleName	A-Z LastName
1	1	0	Mr.	Orlando	W.	Gee
2	2	0	Mr.	Keith	W.	Harris
3	3	0	Ms.	Donna	W.	Carreras
4	4	0	Ms.	Janet	W.	Gates
5	5	0	Mr.	Lucy	W.	Harrington
6	6	0	Ms.	Rosmarie	W.	Carroll
7	7	0	Mr.	Dominic	W.	Gash

STARTING THE CONNECTORS

```
{
  "name": "AdventureWorksLT2025-cdc",
  "connector": {
    "state": "RUNNING",
    "worker_id": "127.0.1.1:8083",
    "version": "3.2.6-1"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "127.0.1.1:8083",
      "version": "3.2.6-1"
    }
  ],
  "type": "source"
}
```

```
{
  "name": "clickhouse-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "127.0.1.1:8083",
    "version": "v1.3.9"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "127.0.1.1:8083",
      "version": "0.0.1"
    }
  ],
  "type": "sink"
}
```

REAL-TIME PROCESSING EXECUTION (STREAM)

```
$FLINK_HOME/bin/flink run -d -c rtsflink.FlinkKafkaToKafkaFixTimestamp ./flinkjobs/rtsflink-1.0-SNAPSHOT.jar
```

 **Apache Flink Dashboard**

Overview

Jobs

Running Jobs

Completed Jobs

Task Managers

Job Manager

Submit New Job

Version: 2.2.1 | Commit: 450c63e @ 2026-05-03T10:10:44+02:00 | Message: 0

Flink 2.0 Kafka to ClickHouse

Cancel Job

Job ID	24e0451719857608b08cd1a965f90fa8	Job State	RUNNING 1	Actions	Job Manager Log
Job Type	STREAMING	Start Time	2026-06-07 18:19:29.439	Duration	1h 41m 30s

[Overview](#) | [Exceptions](#) | [Data Skew](#) | [TimeLine](#) | [Checkpoints](#) | [Configuration](#)

Source: Kafka to ClickHouse -> (Sink: Print to Std. Out, Map -> Sink: Writer -> Sink: Committer)

Parallelism: 1

Backpressured (max): 0%
Busy (max): 0%
Data Skew: 0%

Detail

SubTasks

TaskManagers

Watermarks

Accumulators

Back

...

Subtask Metrics

Aggregated Metrics

ID	Bytes Received	Records Received	Byte	Status	More
0	4.92 MB	7,637	4.76 Mi	RUNNING	...

Sensitivity Label: General

STARTING THE BATCH JOB (BRONZE)

Read from Source

```
# -----  
# READ FROM MSSQL  
# -----  
df = pl.read_database(  
    query=f"SELECT * FROM {source_table}",  
    connection=adbc_cursor,  
    iter_batches=False,  
    batch_size=BATCH_SIZE,  
    schema_overrides=None,  
    infer_schema_length=100,  
    execute_options=None,  
)  
.with_columns(  
    pl.lit(yesterday).alias("buss_date"),  
    pl.col("rowguid").map_batches(  
        lambda batch: pl.Series(  
            str(uuid.UUID(bytes_le=element))  
            if element is not None  
            else None  
            for element in batch  
        ),  
        return_dtype=pl.String  
    ).str.to_uppercase().alias("rowguidcast")  
)  
.drop("rowguid")
```

Write to Bronze Table

```
# -----  
# WRITE TO ICEBERG  
# -----  
if mode == "overwrite":  
    df.write_iceberg(  
        target=target,  
        mode="overwrite",  
    )  
else:  
    df.write_iceberg(  
        target=target,  
        mode="append",  
    )  
  
catalog.close()  
adbc_cursor.close()  
adbc_conn.close()
```

BRONZE ZONE TRANSFORMATION OUTPUT (BATCH)

```
$ duckdb
DuckDB v1.5.2 (Variegata)
Enter ".help" for usage hints.
memory D LOAD https;
memory D LOAD iceberg;
memory D CREATE SECRET (
  TYPE s3,
  KEY_ID 'minioadmin',
  SECRET 'minioadmin123',
  REGION 'ap-southeast-1',
  ENDPOINT 'localhost:9100',
  URL_STYLE 'path',
  USE_SSL false
);
```

Success

boolean

true

```
memory D SELECT * FROM iceberg_scan('s3://lakehouse-staging/bronzebatchlayer/customer/metadata/00001-09ff0218-aa68-4e39-a97f-bfcd9a999085.metadata.json') LIMIT 5;
```

buss_date date	CustomerID int32	NameStyle boolean	Title varchar	FirstName varchar	...	Phone varchar	PasswordHash varchar	PasswordSalt varchar	rowguid varchar	ModifiedDate timestamp with time...
2026-06-06	1	false	Mr.	Orlando	...	245-555-0173	L/RLwxzp4w7RWmEgXX+/A7cXa...	1KjXYs4=	3F5AE95E-B87D-4AED-...	2026-06-04 06:58:36...
2026-06-06	2	false	Mr.	Keith	...	170-555-0127	YPdtRdvqeAhj6wyxEsFdshBDN...	fs1ZGhY=	E552F657-A9AF-4A7D-...	2026-06-04 06:58:36...
2026-06-06	3	false	Ms.	Donna	...	279-555-0130	LNoK27abGQo48gGue3EBV/Url...	YTNH5Rw=	130774B1-DB21-4EF3-...	2026-06-04 06:58:36...
2026-06-06	4	false	Ms.	Janet	...	710-555-0173	ElzTpSNbUW1Ut+L5cWlFR7MF6...	nm7D5e4=	FF862851-1DAA-4044-...	2026-06-04 06:58:36...
2026-06-06	5	false	Mr.	Lucy	...	828-555-0186	KJqV15wsX3PG8TS5GSddp6LFF...	cNFKU4w=	83905BDC-6F5E-4F71-...	2026-06-04 06:58:36...

5 rows
memory D

use .last to show entire result

16 columns (10 shown)

Note:

- Scanning an Iceberg table is performed through the current metadata file, although data files can also be scanned directly
- Bronze zone tables contain data for a single date period

BRONZE ZONE TRANSFORMATION OUTPUT (BATCH)

The screenshot displays the Minio Object Store interface. On the left, a sidebar contains the 'OBJECT STORE Community Edition' logo, a 'Create Bucket' button, a 'Filter Buckets' search bar, and a list of buckets: 'lakehouse', 'lakehouse-rest', 'lakehouse-staging', and 'lakehouse-transform'. Below the bucket list are links for 'Documentation', 'License', and 'Sign Out'. The main area is titled 'Object Browser' and features a search bar with the placeholder 'Start typing to filter objects in the bucket'. The selected bucket is 'lakehouse-staging', which was created on 'Fri, May 29 2026 21:17:44 (GMT+7)', has 'PRIVATE' access, and contains 47.4 MiB of data across 624 objects. Action buttons for 'Rewind', 'Refresh', and 'Upload' are visible. The breadcrumb path is 'lakehouse-staging / bronzebatchlayer / customer / data'. A table lists the objects in this path, showing their names, last modified times, and sizes.

☐	Name	Last Modified	Size
☐	00000-0-0d7e1ca7-0f3c-43aa-933c-1de91a8fa...	Today, 00:24	67.6 KiB
☐	00000-0-2cf285b5-f556-45cd-8dba-ecceff017...	Today, 00:28	61.3 KiB
☐	00000-0-5419c32e-1f2b-494f-8b2f-e21b86b2...	Today, 00:37	62.7 KiB
☐	00000-0-5ca5ef2f-0c5f-43e0-9422-8a86f780...	Today, 00:54	62.6 KiB
☐	00000-0-6f56fe66-9f11-444d-9c83-c244a7ae...	Today, 00:36	62.7 KiB

Note:

- Data files use the .parquet extension
- Metadata files use the .json extension

STARTING THE BATCH JOB (SILVER)

Read from Bronze Table

```
# -----  
# READ FROM ICEBERG  
# -----  
lf = pl.scan_iceberg(  
    source_table,  
    catalog=catalog,  
    reader_override="pyiceberg",  
    use_metadata_statistics=True,  
    fast_deletion_count=None,  
    use_pyiceberg_filter=True,  
)
```

Merge into Silver Table

```
# -----  
# WRITE TO ICEBERG (UPSERT)  
# -----  
for batch_number, batch in enumerate(  
    lf.collect_batches(engine="streaming", chunk_size=BATCH_SIZE,),  
    start=1,  
):  
    target.upsert(  
        df=batch.to_arrow(),  
        join_cols=["buss_date", "rowguid"],  
    )  
  
    batch_rows = len(batch)  
    total_rows += batch_rows  
    print(  
        f"Batch #{batch_number} | "  
        f"rows = {batch_rows} | "  
        f"total = {total_rows}"  
    )
```

SILVER ZONE TRANSFORMATION OUTPUT (BATCH)

```
memory D SELECT DISTINCT buss_date FROM iceberg_scan('s3://lakehouse-transform/silverbatchlayer/customer/metadata/00013-613c4052-1547-4d67-9a3c-52b0a34b7dc2.metadata.json') LIMIT 10;
```

buss_date date
2026-07-10
2026-06-06

```
memory D SELECT * FROM iceberg_scan('s3://lakehouse-transform/silverbatchlayer/customer/metadata/00013-613c4052-1547-4d67-9a3c-52b0a34b7dc2.metadata.json') LIMIT 10;
```

buss_date date	CustomerID int32	NameStyle boolean	Title varchar	...	PasswordSalt varchar	rowguid varchar	ModifiedDate timestamp with time zo...
2026-07-10	29583	false	Mr.	...	6ZoXk8E=	63797367-055A-48FC-97C0-2C4D8573585E	2026-02-24 07:00:00+07
2026-07-10	29584	false	Mr.	...	PiILPfc=	BD077ADD-F8F5-471C-A5FD-31A61303F41F	2026-02-24 07:00:00+07
2026-07-10	29585	false	Mr.	...	J9DNyhU=	11E65E98-1980-4D9F-A6C0-A6DD2885B673	2026-02-24 07:00:00+07
2026-07-10	29587	false	Mr.	...	C40iUF4=	8916316F-55E7-453E-9C82-F59B5FC32913	2026-02-24 07:00:00+07
2026-07-10	29588	false	Mr.	...	dS1y3X0=	DEB25F8C-1D9F-442A-B686-6A505DE0C1BF	2026-02-24 07:00:00+07
2026-07-10	29590	false	Mr.	...	T2LZOPo=	7E529A97-2B5C-4339-883F-3ECCFB85E411	2026-02-24 07:00:00+07
2026-07-10	29591	false	Mr.	...	mQK+bIw=	74AA6608-703F-4F37-A447-BBF9879C0D4F	2026-02-24 07:00:00+07
2026-07-10	29593	false	Ms.	...	dt/JUB4=	C610F872-6183-4D98-ADA3-E3A5B1A3588D	2026-02-24 07:00:00+07
2026-07-10	29596	false	Mr.	...	MA0hXKA=	A6172F3E-C548-45CB-B1D7-79CA27AC267B	2026-02-24 07:00:00+07
2026-07-10	29598	false	Mr.	...	MjQeaAE=	5411839C-DAA1-4827-A995-1856C8217DE2	2026-02-24 07:00:00+07

10 rows use .last to show entire result 16 columns (7 shown)
memory D

Note:

- Silver zone tables contain historical data
- For convenience, staging and historical tables have the same structure
- Add any processes as needed (e.g., cleansing, enrichment, etc.)

STARTING THE BATCH JOB (GOLDEN)

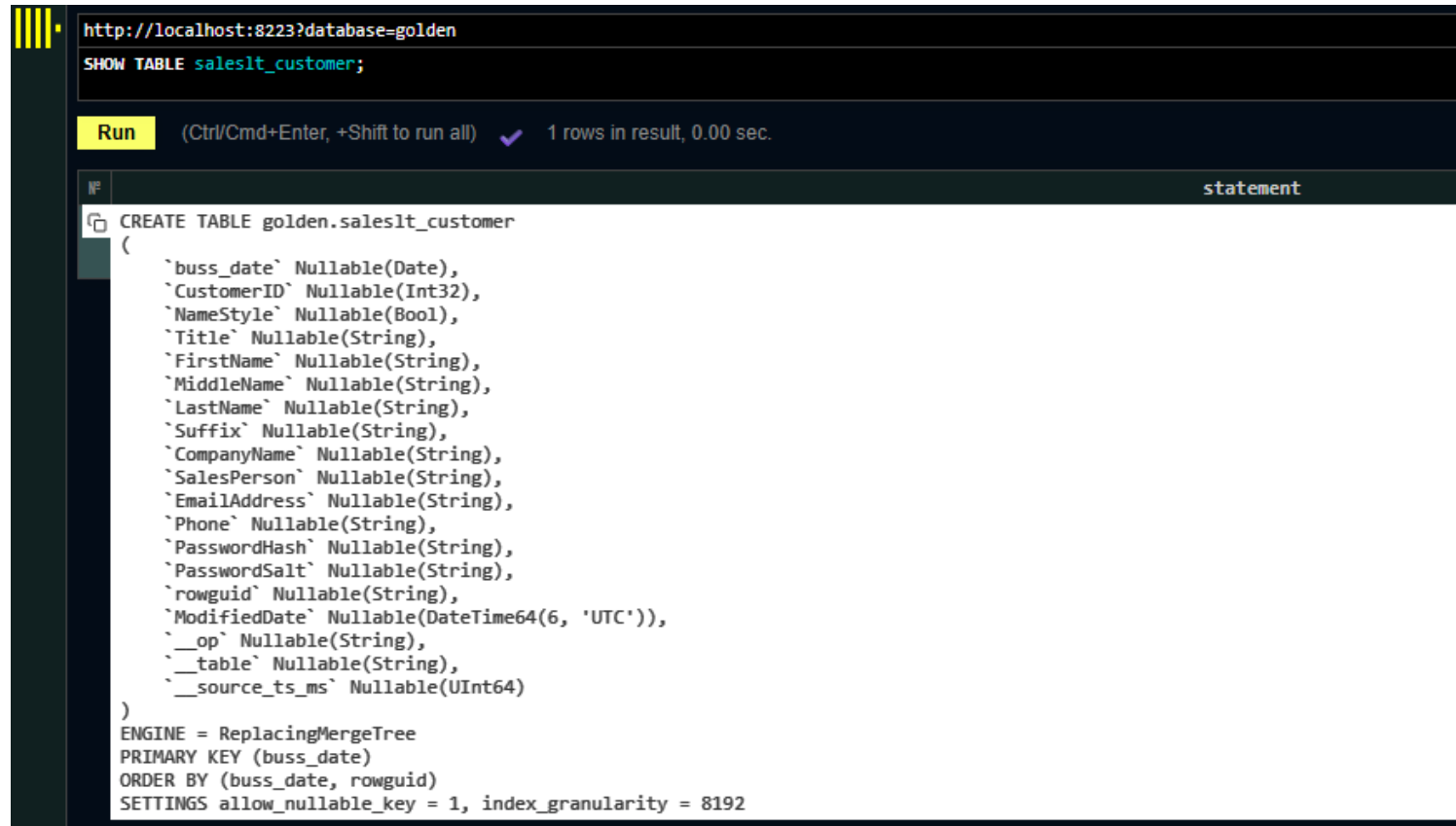
Read from Silver Table

```
# -----  
# READ FROM ICEBERG  
# -----  
lf = pl.scan_iceberg(  
    source_table,  
    catalog=catalog,  
    reader_override="pyiceberg",  
    use_metadata_statistics=True,  
    fast_deletion_count=None,  
    use_pyiceberg_filter=True,  
)  
.filter(  
    pl.col("buss_date").dt.date() == yesterday  
)
```

Write to ClickHouse

```
# -----  
# WRITE TO CLICKHOUSE  
# -----  
for batch_number, batch in enumerate(  
    lf.collect_batches(engine="streaming", chunk_size=BATCH_SIZE,),  
    start=1,  
):  
    batch.write_database(  
        table_name=target_table,  
        connection=conn,  
        if_table_exists="append",  
        engine="sqlalchemy"  
    )  
  
    batch_rows = len(batch)  
    total_rows += batch_rows  
    print(  
        f"Batch #{batch_number} | "  
        f"rows = {batch_rows} | "  
        f"total = {total_rows}"  
    )
```

CUSTOMER TABLE USES STATE MODEL



http://localhost:8223?database=golden

SHOW TABLE saleslt_customer;

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 1 rows in result, 0.00 sec.

#	statement
1	<pre>CREATE TABLE golden.saleslt_customer (`buss_date` Nullable(Date), `CustomerID` Nullable(Int32), `NameStyle` Nullable(Bool), `Title` Nullable(String), `FirstName` Nullable(String), `MiddleName` Nullable(String), `LastName` Nullable(String), `Suffix` Nullable(String), `CompanyName` Nullable(String), `SalesPerson` Nullable(String), `EmailAddress` Nullable(String), `Phone` Nullable(String), `PasswordHash` Nullable(String), `PasswordSalt` Nullable(String), `rowguid` Nullable(String), `ModifiedDate` Nullable(DateTime64(6, 'UTC')), `__op` Nullable(String), `__table` Nullable(String), `__source_ts_ms` Nullable(UInt64)) ENGINE = ReplacingMergeTree PRIMARY KEY (buss_date) ORDER BY (buss_date, rowguid) SETTINGS allow_nullable_key = 1, index_granularity = 8192</pre>

Note:

- Reduce Nullable() usage where possible; this is for testing purposes

TABLE CHECK



http://localhost:8223?database=goldenv26.6.1.272, uptime 2 hr • defaultpassword

```
SELECT 'saleslt_productmodel', COUNT(1) ROW_COUNT FROM golden.saleslt_productmodel UNION ALL
SELECT 'saleslt_customeraddress', COUNT(1) ROW_COUNT FROM golden.saleslt_customeraddress UNION ALL
SELECT 'saleslt_productmodelproductdescription', COUNT(1) ROW_COUNT FROM golden.saleslt_productmodelproductdescription UNION ALL
SELECT 'saleslt_salesorderheader', COUNT(1) ROW_COUNT FROM golden.saleslt_salesorderheader UNION ALL
SELECT 'saleslt_customer', COUNT(1) ROW_COUNT FROM golden.saleslt_customer UNION ALL
SELECT 'saleslt_address', COUNT(1) ROW_COUNT FROM golden.saleslt_address UNION ALL
SELECT 'saleslt_productdescription', COUNT(1) ROW_COUNT FROM golden.saleslt_productdescription UNION ALL
SELECT 'saleslt_salesorderdetail', COUNT(1) ROW_COUNT FROM golden.saleslt_salesorderdetail UNION ALL
SELECT 'saleslt_productcategory', COUNT(1) ROW_COUNT FROM golden.saleslt_productcategory UNION ALL
SELECT 'saleslt_product', COUNT(1) ROW_COUNT FROM golden.saleslt_product;
```

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 10 rows in result, 0.01 sec. Read 10 rows, 160.00 B  

Nº	'saleslt_productmodel'	ROW_COUNT
1	saleslt_address	2250
2	saleslt_productmodel	640
3	saleslt_customeraddress	2085
4	saleslt_productmodelproductdescription	3048
5	saleslt_salesorderheader	160
6	saleslt_customer	2146
7	saleslt_productdescription	3048
8	saleslt_salesorderdetail	2168
9	saleslt_productcategory	205



CUSTOMER TABLE RESULT #1, INITIAL STATE

http://localhost:8223?database=goldenv26.6.1.272, uptime 20 hr • defaultpassword

SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer WHERE CustomerID = '30016'

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 2 rows in result, 0.02 sec.100.0%, Read 501 rows, 45.61 KB

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	u	Customer	1783768752523
2	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL

ClickHouse

CUSTOMER TABLE RESULT #2, END-OF-DAY BATCH JOB EXECUTION

http://localhost:8223?database=goldenv26.6.1.272, uptime 20 hrdefaultpassword

SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer WHERE CustomerID = '30016'

Run (Ctrl/Cmd+Enter, +Shift to run all) 3 rows in result, 0.03 sec. 100.0%, Read 1.70 thousand rows, 124.51 KB

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
2	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	u	Customer	1783768752523
3	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL

ClickHouse

CUSTOMER TABLE RESULT #3, STREAM CAPTURES DATA ON THE NEXT DAY

http://localhost:8223?database=goldenv26.6.1.272, uptime 20 hrdefaultpassword

SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer WHERE CustomerID = '30016'

Run (Ctrl/Cmd+Enter, +Shift to run all) 4 rows in result, 0.01 sec.100.0%, Read 1.35 thousand rows, 122.88 KB

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
2	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL
3	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	u	Customer	1783768752523
4	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	KL.	u	Customer	1783789909457

ClickHouse

CUSTOMER TABLE RESULT #4, ROWS COLLAPSED

http://localhost:8223?database=goldenv26.6.1.272, uptime 20 hrdefaultpassword

```
SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer WHERE CustomerID = '30016'
```

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 3 rows in result, 0.01 sec.100.0%, Read 1.70 thousand rows, 154.44 KB

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
2	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL
3	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	KL.	u	Customer	1783789909457

ClickHouse

CUSTOMER TABLE RESULT #5, HANDLE MULTIPLE CHANGES ON THE SAME DAY

http://localhost:8223?database=goldenv26.6.1.272, uptime 20 hrdefaultpassword

```
SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer WHERE CustomerID = '30016'
```

Run (Ctrl/Cmd+Enter, +Shift to run all) 5 rows in result, 0.01 sec. 100.0%, Read 1.70 thousand rows, 154.64 KB

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	A.	u	Customer	1783790245153
2	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
3	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL
4	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	Q.	u	Customer	1783790254810
5	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	KL.	u	Customer	1783789909457

CUSTOMER TABLE RESULT #6, FINAL RESULT

http://localhost:8223?database=goldenv26.6.1.272, uptime 20 hr • defaultpassword

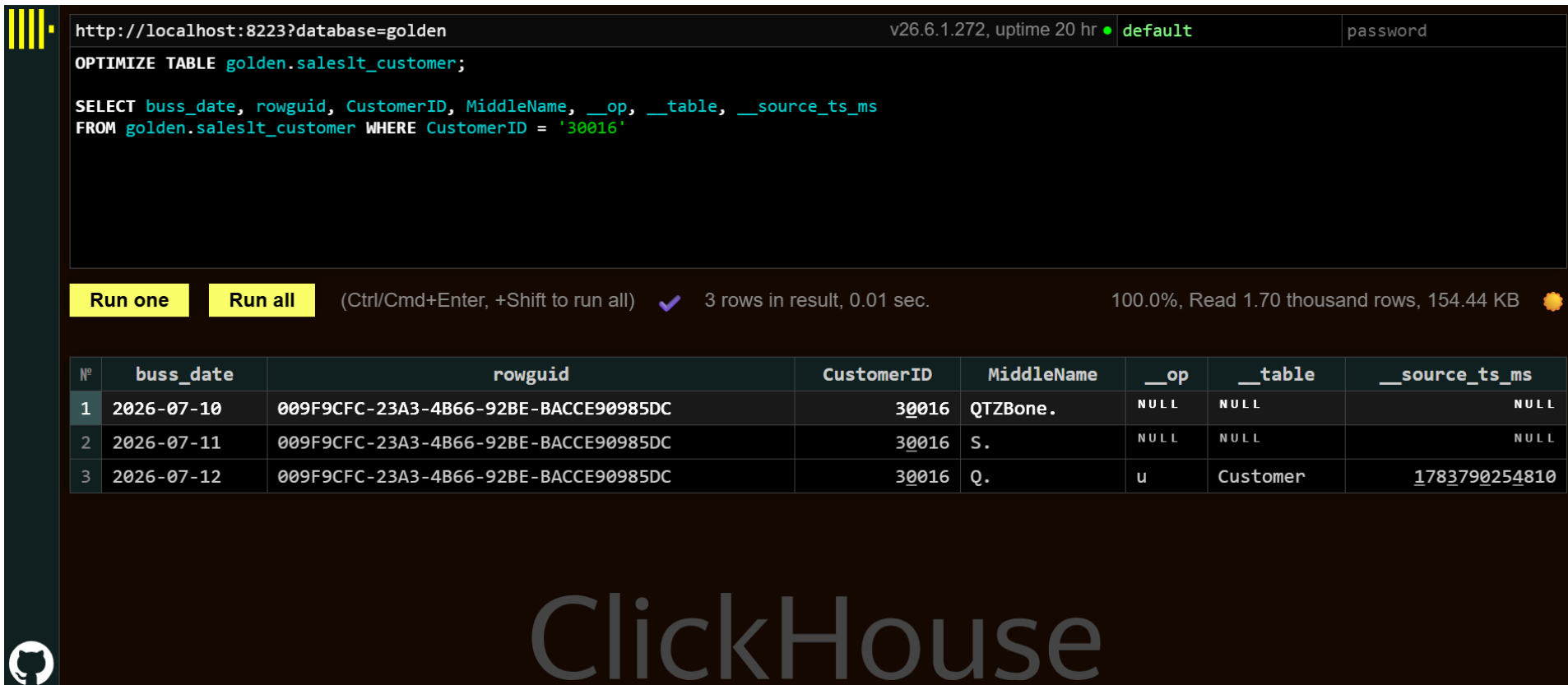
SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer FINAL WHERE CustomerID = '30016'

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 3 rows in result, 0.01 sec.100.0%, Read 1.70 thousand rows, 154.64 KB

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
2	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL
3	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	Q.	u	Customer	1783790254810

ClickHouse

CUSTOMER TABLE RESULT #7, MANUAL MERGE HINT



http://localhost:8223?database=golden v26.6.1.272, uptime 20 hr • default password

```
OPTIMIZE TABLE golden.saleslt_customer;
```

```
SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms  
FROM golden.saleslt_customer WHERE CustomerID = '30016'
```

Run one **Run all** (Ctrl/Cmd+Enter, +Shift to run all) ✓ 3 rows in result, 0.01 sec. 100.0%, Read 1.70 thousand rows, 154.44 KB

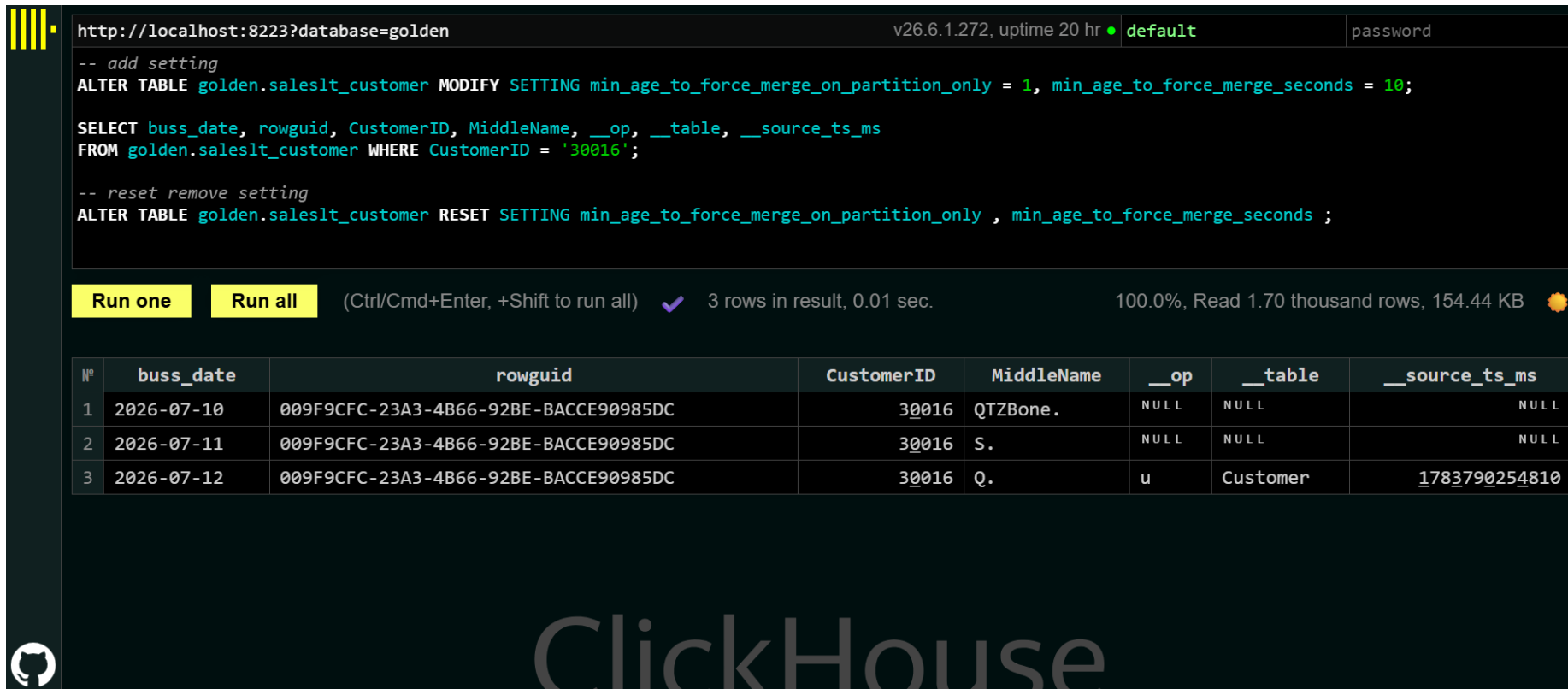
Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
2	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL
3	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	Q.	u	Customer	1783790254810

ClickHouse

Note:

- Unscheduled merges may cause resource contention and impact query performance, especially on heavily used tables with high user traffic

CUSTOMER TABLE RESULT #8, FORCE MERGE AFTER A SPECIFIC PERIOD



The image shows a ClickHouse SQL client interface. At the top, the URL is `http://localhost:8223?database=golden`, the version is `v26.6.1.272`, uptime is `20 hr`, and the user is `default` with a password field. The SQL query executed is:

```
-- add setting
ALTER TABLE golden.saleslt_customer MODIFY SETTING min_age_to_force_merge_on_partition_only = 1, min_age_to_force_merge_seconds = 10;

SELECT buss_date, rowguid, CustomerID, MiddleName, __op, __table, __source_ts_ms
FROM golden.saleslt_customer WHERE CustomerID = '30016';

-- reset remove setting
ALTER TABLE golden.saleslt_customer RESET SETTING min_age_to_force_merge_on_partition_only , min_age_to_force_merge_seconds ;
```

Below the query, there are buttons for **Run one** and **Run all**, and a status bar indicating **3 rows in result, 0.01 sec.** and **100.0%, Read 1.70 thousand rows, 154.44 KB**.

Nº	buss_date	rowguid	CustomerID	MiddleName	__op	__table	__source_ts_ms
1	2026-07-10	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	QTZBone.	NULL	NULL	NULL
2	2026-07-11	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	S.	NULL	NULL	NULL
3	2026-07-12	009F9CFC-23A3-4B66-92BE-BACCE90985DC	30016	Q.	u	Customer	1783790254810

The ClickHouse logo is visible in the bottom left corner.

Note:

- ClickHouse default configuration is good enough, so this parameter likely does not need to be changed

THE SAME IDEA IN AGGREGATION

```
$FLINK_HOME/bin/flink run -d -c rtsflink.FlinkAggregation ./flinkjobs/rtsflink-1.0-SNAPSHOT.jar
```

Apache Flink Dashboard

Overview

Jobs

Running Jobs

Completed Jobs

Task Managers

Job Manager

Submit New Job

Flink 2.0 Kafka to ClickHouse Aggregation

Cancel Job

Job ID	536f0a529843c5242e6854568db81a58	Job State	RUNNING 2	Actions	Job Manager Log
Job Type	STREAMING	Start Time	2026-07-10 22:15:32.906	Duration	14h 16m 55s

OverviewExceptionsData SkewTimeLineCheckpointsConfiguration

ClickHouse Aggregation - Sink: Print to Std. Out

Parallelism: 1

Backpressed (max): 0%

Busy (max): 2%

Data Skew: 0%

Low Watermark: 1783745052550

HASH

TumblingEventTimeWindows - Sink: Writer -> Sink: Committer

Parallelism: 1

Backpressed (max): 0%

Busy (max): 2%

Data Skew: 0%

Low Watermark: 1783745052550

DetailSubTasksTaskManagersWatermarksAccumulatorsBackPressureMetricsFlare...

Window(TumblingEventTimeWindows(30000), EventTimeTrigger, ProductModelAggregate, ProductModelProcessWindowFunction)

+ Sink: Writer

+ Sink: Committer

State: **RUNNING**

Task: 1

Parallelism: 1

Records Sent: 12

Max Parallelism: 128

Records Received: 42

Start Time: 2026-07-10 22:15:33.707

Bytes Sent: 8.59 KB

End Time: -

Bytes Received: 5.24 KB

Duration: 14h 16m 54s

Name	Status	Bytes Received	Records Received	Bytes Sent	Records Sent	Parallelism	Tasks
Source: Kafka to ClickHouse Aggregation -> Sink: Print to Std. Out	RUNNING	15.5 KB	42	512 B	42	1	1
TumblingEventTimeWindows -> Sink: Writer -> Sink: Committer	RUNNING	5.24 KB	42	8.59 KB	12	1	1

THE SAME IDEA IN AGGREGATION

http://localhost:8223?database=goldenv26.6.1.272, uptime 15 mindefaultpassword

```
SELECT * FROM golden.productmodelagg;

/*
CREATE TABLE golden.productmodelagg
(
    `buss_date` Nullable(Date),
    `ProductModelID` Nullable(Int32),
    `SumID` Nullable(Int32),
    `DistinctCountRowguid` Nullable(Int32)
)
ENGINE = ReplacingMergeTree
PRIMARY KEY (buss_date)
ORDER BY (buss_date, ProductModelID)
SETTINGS allow_nullable_key = 1, index_granularity = 8192
*/
```

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 7 rows in result, 0.01 sec. 100.0%, Read 7 rows, 126.00 B

Nº	buss_date	ProductModelID	SumID	DistinctCountRowguid
1	2026-07-10	119	833	7
2	2026-07-10	122	244	1
3	2026-07-10	124	124	1
4	2026-07-10	125	125	1
5	2026-07-10	126	126	1
6	2026-07-10	127	127	1
7	2026-07-10	128	128	1

INTENTIONAL MISCONFIGURED STATE MANAGEMENT RESULTS IN INCORRECT RESULTS

```
s.AdventureWorksLT2025.SalesLT.ProductModel --property print.key=true --property print.value=true --property key.separator="
| "
Option --property is deprecated and will be removed in a future version. Use --formatter-property instead.
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.
apache.org/documentation/#consumer_rebalance_protocol
128 | {"buss_date": "2026-07-10", "ProductModelID": 128, "SumID": 128, "DistinctCountRowguid": 1}
126 | {"buss_date": "2026-07-10", "ProductModelID": 126, "SumID": 126, "DistinctCountRowguid": 1}
125 | {"buss_date": "2026-07-10", "ProductModelID": 125, "SumID": 125, "DistinctCountRowguid": 1}
124 | {"buss_date": "2026-07-10", "ProductModelID": 124, "SumID": 124, "DistinctCountRowguid": 1}
127 | {"buss_date": "2026-07-10", "ProductModelID": 127, "SumID": 127, "DistinctCountRowguid": 1}
128 | {"buss_date": "2026-07-10", "ProductModelID": 128, "SumID": 128, "DistinctCountRowguid": 1}
126 | {"buss_date": "2026-07-10", "ProductModelID": 126, "SumID": 126, "DistinctCountRowguid": 1}
125 | {"buss_date": "2026-07-10", "ProductModelID": 125, "SumID": 125, "DistinctCountRowguid": 1}
124 | {"buss_date": "2026-07-10", "ProductModelID": 124, "SumID": 124, "DistinctCountRowguid": 1}
128 | {"buss_date": "2026-07-10", "ProductModelID": 128, "SumID": 512, "DistinctCountRowguid": 4}
128 | {"buss_date": "2026-07-10", "ProductModelID": 128, "SumID": 128, "DistinctCountRowguid": 1}
122 | {"buss_date": "2026-07-10", "ProductModelID": 122, "SumID": 854, "DistinctCountRowguid": 1}
122 | {"buss_date": "2026-07-10", "ProductModelID": 122, "SumID": 244, "DistinctCountRowguid": 1}
119 | {"buss_date": "2026-07-10", "ProductModelID": 119, "SumID": 833, "DistinctCountRowguid": 7}
119 | {"buss_date": "2026-07-10", "ProductModelID": 119, "SumID": 119, "DistinctCountRowguid": 1}
116 | {"buss_date": "2026-07-11", "ProductModelID": 116, "SumID": 464, "DistinctCountRowguid": 1}
116 | {"buss_date": "2026-07-11", "ProductModelID": 116, "SumID": 348, "DistinctCountRowguid": 1}
116 | {"buss_date": "2026-07-11", "ProductModelID": 116, "SumID": 116, "DistinctCountRowguid": 1}
100 | {"buss_date": "2026-07-11", "ProductModelID": 100, "SumID": 500, "DistinctCountRowguid": 1}
116 | {"buss_date": "2026-07-11", "ProductModelID": 116, "SumID": 116, "DistinctCountRowguid": 1}
116 | {"buss_date": "2026-07-11", "ProductModelID": 116, "SumID": 464, "DistinctCountRowguid": 1}
```

Note:

- The job is continuously executed using a minute-level processing window, but the data model is designed for daily time-indexed aggregation

DATA INCONSISTENCIES CAUSED BY MISCONFIGURED STATE MANAGEMENT

http://localhost:8223?database=golden v26.6.1.272, uptime 15 min default password

```
SELECT * FROM golden.productmodelagg;
```

```
/*  
CREATE TABLE golden.productmodelagg  
(  
    `buss_date` Nullable(Date),  
    `ProductModelID` Nullable(Int32),  
    `SumID` Nullable(Int32),  
    `DistinctCountRowguid` Nullable(Int32)  
)  
ENGINE = ReplacingMergeTree  
PRIMARY KEY (buss_date)  
ORDER BY (buss_date, ProductModelID)  
SETTINGS allow_nullable_key = 1, index_granularity = 8192  
*/
```

Run (Ctrl/Cmd+Enter, +Shift to run all) ✓ 10 rows in result, 0.02 sec. 100.0%, Read 10 rows, 180.00 B

Nº	buss_date	ProductModelID	SumID	DistinctCountRowguid
1	2026-07-10	119	119	1
2	2026-07-10	122	244	1
3	2026-07-10	124	124	1
4	2026-07-10	125	125	1
5	2026-07-10	126	126	1
6	2026-07-10	127	127	1
7	2026-07-10	128	128	1
8	2026-07-11	116	116	1
9	2026-07-11	100	500	1
10	2026-07-11	116	464	1

JOIN ENRICHMENT EXAMPLE

```
$FLINK_HOME/bin/flink run -d -c rtslink.FlinkJoinCoProcess ./flinkjobs/rtslink-1.0-SNAPSHOT.jar
```

Apache Flink Dashboard

Overview

Jobs

Running Jobs

Completed Jobs

Task Managers

Job Manager

Submit New Job

Flink 2.0 Kafka to ClickHouse Join 2 Stream

Cancel Job

Job ID	2b5ce63f7f11569a87969e87fe0d96bc	Job State	RUNNING 3	Actions	Job Manager Log
Job Type	STREAMING	Start Time	2026-07-11 16:48:58.447	Duration	40m 26s 544ms

Overview Exceptions Data Skew TimeLine Checkpoints Configuration

Source: SalesOrderDetail - Kafka to ClickHouse Join Stream Enrichment -> Sink: Print to Std. Out
Parallelism: 1
Backpressured (max): 0%
Busy (max): 3%
Data Skew: 0%

Source: SalesOrderHeader - Kafka to ClickHouse Join Stream Enrichment -> Sink: Print to Std. Out
Parallelism: 1
Backpressured (max): 0%
Busy (max): 3%
Data Skew: 0%

Co-Keyed-Process -> Sink: Writer -> Sink: Committer
Parallelism: 1
Backpressured (max): 0%
Busy (max): 3%
Data Skew: 0%
Low Watermark: 17837650

HASH

HASH

Detail SubTasks TaskManagers Watermarks Accumulators BackPressure Metrics FlameGraph

Co-Keyed-Process
+- Sink: Writer
+- Sink: Committer

State : **RUNNING**

Parallelism : 1

Max Parallelism : 128

Start Time : 2026-07-11 16:49:00.931

End Time : -

Duration : 40m 24s 60ms

Task : **1**

Records Sent : 2

Records Received : 7

Bytes Sent : 2.13 KB

Bytes Received : 1.44 KB

Name	Status	Bytes Received	Records Received	Bytes Sent	Records Sent	Parallelism	Start Time	Tasks
Source: SalesOrderDetail - Kafka to ClickHouse Join Stream Enrichment -> Sink: Print to Std. Out	RUNNING	953 B	2	0 B	2	1	2026-07-11 16:49:00.931	1
Source: SalesOrderHeader - Kafka to ClickHouse Join Stream Enrichment -> Sink: Print to Std. Out	RUNNING	3.57 KB	5	256 B	5	1	2026-07-11 16:49:00.931	1
Co-Keyed-Process -> Sink: Writer -> Sink: Committer	RUNNING	1.44 KB	7	2.13 KB	2	1	2026-07-11 16:49:00.931	1

JOIN ENRICHMENT RESULT

Option --property is deprecated and will be removed in a future version. Use --formatter-property instead.
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.apache.org/documentation/#consumer_rebalance_protocol

```
113409 | {"buss_date":"2026-07-11","SalesOrderID":71947,"SalesOrderDetailID":113409,"OrderQty":1,"ProductID":916,"UnitPrice":400.0,"UnitPriceDiscount":0.0,"LineTotal":0.0,"OrderDate":"1212278400000","DueDate":"1213315200000","ShipDate":"1212883200000","SalesOrderNumber":"","PurchaseOrderNumber":null,"AccountNumber":null,"ShipMethod":"CARGO TRANSPORT 5","rowguid":"C75DA746-34F1-4AAC-92FC-1E5A26876B09","ModifiedDate":"2026-04-30 00:00:00.000","__op":"c","__table":"SalesOrderDetail","__source_ts_ms":1783765076960}
113410 | {"buss_date":"2026-07-11","SalesOrderID":71947,"SalesOrderDetailID":113410,"OrderQty":1,"ProductID":916,"UnitPrice":338.9536,"UnitPriceDiscount":0.0,"LineTotal":0.0,"OrderDate":"1212278400000","DueDate":"1213315200000","ShipDate":"1212883200000","SalesOrderNumber":"","PurchaseOrderNumber":null,"AccountNumber":null,"ShipMethod":"CARGO TRANSPORT 5","rowguid":"19958B50-BCFC-4526-9343-C97B38BB2316","ModifiedDate":"2026-04-30 00:00:00.000","__op":"c","__table":"SalesOrderDetail","__source_ts_ms":1783765076963}
```

JOIN ENRICHMENT RESULT

http://localhost:8223?database=golden

v26.6.1.272, uptime 15 min

default

password

```
CREATE TABLE golden.enrichedsalesorder
(
  `buss_date` Nullable(Date),
  `SalesOrderID` Nullable(Int32),
  `SalesOrderDetailID` Nullable(Int32),
  `OrderQty` Nullable(Int32),
  `ProductID` Nullable(Int32),
  `UnitPrice` Nullable(Decimal(19, 4)),
  `UnitPriceDiscount` Nullable(Decimal(19, 4)),
  `LineTotal` Nullable(Decimal(38, 6)),
  `OrderDate` Nullable(DateTime64(6, 'UTC')),
  `DueDate` Nullable(DateTime64(6, 'UTC')),
  `ShipDate` Nullable(DateTime64(6, 'UTC')),
  `SalesOrderNumber` Nullable(String),
  `PurchaseOrderNumber` Nullable(String),
  `AccountNumber` Nullable(String),
  `ShipMethod` Nullable(String),
  `rowguid` Nullable(String),
  `ModifiedDate` Nullable(DateTime64(6, 'UTC')),
  `__op` Nullable(String),
  `__table` Nullable(String),
  `__source_ts_ms` Nullable(UInt64)
)
ENGINE = ReplacingMergeTree
PRIMARY KEY (buss_date)
ORDER BY (buss_date, rowguid)
SETTINGS allow_nullable_key = 1, index_granularity = 8192;

SELECT * FROM golden.enrichedsalesorder;
```

Run selected Run all (Ctrl/Cmd+Enter, +Shift to run all) 2 rows in result, 0.00 sec. 100.0%, Read 2 rows, 504.00 B

#	buss_date	SalesOrder...	SalesOrderDetail...	OrderQ...	Product...	UnitPri...	UnitPriceDiscou...	LineTot...	OrderDate	DueDate	ShipDate
1	2026-07-11	71947	113410	1	916	338.9536	0	0	2008-06-01 00:00:00.000000	2008-06-13 00:00:00.000000	2008-06-08 00:00:00.000000
2	2026-07-11	71947	113409	1	916	400	0	0	2008-06-01 00:00:00.000000	2008-06-13 00:00:00.000000	2008-06-08 00:00:00.000000

SUMMARY

ClickHouse ReplacingMergeTree simplifies the processing layer implementation

Lambda act as a bridge/transition toward fully real-time

Immediate aggregation can be used for initial approximation, while historical aggregation ensures completeness

Maintaining dual codebases introduces additional complexity

Incorrect logic and/or configuration can result in data inconsistencies or double counting

Ensuring consistency is the biggest challenge

THANK YOU